

Project Timeline — Minorities Early-Warning System

Twelve-Week Build Log + What Was Delivered Each Phase

Internship Project — Minorities Early-Warning System

June 2026

Contents

What This Document Is	2
Quick Calendar View	2
Phase 1 — Week 1 — Building the Data Pipeline	3
Goal of the phase	3
What we did, step by step	3
Step 1.1 — Explored the source	3
Step 1.2 — Designed the filter	3
Step 1.3 — Wrote the build script	3
Step 1.4 — Found and fixed a critical bug	3
Step 1.5 — Documented	4
What was delivered	4
Headline number from this phase	4
Phase 2 — Weeks 2–3 — Exploratory Data Analysis (EDA)	4
Goal of the phase	4
What we did	4
Step 2.1 — Ran a standard EDA script	4
Step 2.2 — Built the district severity CSV	4
Step 2.3 — Documented findings	5
What was delivered	5
Headline finding	5
Phase 3 — Weeks 3–4 — Interactive Heatmaps	5
Goal of the phase	5
What we did	5
Step 3.1 — Chose a tech stack	5
Step 3.2 — Wrote <code>code_reference/viz/heatmap.py</code>	5
What was delivered	6
Phase 4 — Weeks 5–6 — Feature Engineering	6
Goal of the phase	6
What we did	6
Step 4.1 — Built each family one at a time	6
The six feature families	6

Step 4.2 — Wrote the merge script	6
Headline findings from this phase	6
Phase 5 — Weeks 7–8 — Prediction Models	7
Goal of the phase	7
What we built — three models	7
Model 1 — Per-case classifier (LR + RF)	7
Model 2 — PS-week forecaster (LR)	7
Model 3 — Volume forecaster (Prophet)	7
What was delivered	7
Phase 6 — Weeks 9–10 — The Dashboard	8
Goal of the phase	8
What we built — two versions	8
Version 1 — Self-contained HTML	8
Version 2 — Streamlit web app (public deployment)	8
Version 3 — The “predictions-first” visual rebuild	9
Phase 7 — Weeks 11–12 — Documentation + Delivery	9
Goal of the phase	9
What we did	9
Step 7.1 — System audit	9
Step 7.2 — Privacy hardening	9
Step 7.3 — Portable bundle	9
Step 7.4a — Public deployment + intern handoff	9
Step 7.4 — Five formal deliverables	10
What was delivered	10
Cross-Reference — What File Came From Which Phase	10
What’s Been Delivered	11

What This Document Is

This is the **project timeline** — a step-by-step record of what was built each week, from kickoff to the final documentation phase. If you are new to project documentation, the timeline serves four purposes:

1. **Tells you the order things happened in.** Each week builds on the previous one.
2. **Tells you what was delivered.** Every week ends with concrete artifacts you can actually point at (a file, a chart, a table).
3. **Helps anyone repeat the project.** Read week-by-week, follow the commands, get the same outputs.
4. **Shows where time was spent.** Some weeks needed more work than the plan suggested; others were quicker. This is useful for anyone planning a similar project in the future.

A 12-week plan from the original PSCA brief was the starting point. This document updates that plan with what was *actually* built.

Quick Calendar View

WEEK 1	Data pipeline	minority_incidents (4,110 rows)
--------	---------------	---------------------------------

WEEK 2-3	EDA + heatmaps	8 figures, district severity, 3 HTML heatmaps
WEEK 5-6	Feature engineering	6 feature families, merged 62-column table
WEEK 7-8	Modeling	3 models (per-case classifier, PS-week forecaster, Proph
WEEK 9-10	Dashboard	HTML dashboard + predictions-first visual rebuild
WEEK 11-12	Documentation + handoff	6 deliverables + public Streamlit deployment

Phase 1 — Week 1 — Building the Data Pipeline

Goal of the phase

Get the data out of PSCA’s `<source_call_log_table>` table (which has 5.7 million rows of every Emergency-15 call) and into a focused working table called `minority_incidents` (which has only the calls that involve a religious minority — about 4,110 rows).

What we did, step by step

Step 1.1 — Explored the source

We connected to the `<source_db>` database and looked at the columns of `<source_call_log_table>`. We saw: - A 3-level case-nature hierarchy with a `Religious Offences` Level-2 category. - A free-text `description` field where the caller’s narrative is stored. - Location columns (lat, long). - A 3-year window of data (2024–2026).

Step 1.2 — Designed the filter

We chose a **two-clause filter** for picking out minority-related calls:

Keep a call if EITHER: (a) The Level-2 case category is `Religious Offences`, OR (b) The description text mentions a minority community (Christian, Ahmadi, Hindu, Sikh), a place of worship (church, gurdwara, temple), or blasphemy.

This is intentionally wide. We’d rather over-collect at this layer than miss real cases.

Step 1.3 — Wrote the build script

`code_reference/data/build_minority_incidents.py` does the following:

1. Reads from `<source_call_log_table>` with the filter.
2. Looks up district names from `<station_boundaries_ref>`.
3. Derives extra fields: `is_lahore`, `minority_community` (from description regex), `is_minority_targeted` (the strict label), `match_source` (which filter clause hit).
4. Writes the result into `<analytics_db>.minority_incidents`.

Step 1.4 — Found and fixed a critical bug

Initially the table came out with **every incident_date and <received_time> set to NULL**. The bug was that the source columns are text strings, not native dates, and the Python `try/except` block swallowed the parse error.

We fixed it by replacing the brittle date conversion with an explicit `datetime.strptime` that tries multiple known formats. Counts didn’t change (still 4,110 rows), but the date columns now work correctly, which unblocks all the downstream feature work.

Step 1.5 — Documented

Created two documentation files:

- <see project phase docs> — every column explained
- <see project phase docs> — what the build does and why

What was delivered

Artifact	What
Database table	<code>minority_incidents</code> — 4,110 rows
Schema file	<code>code_reference/data/schema.sql</code>
Build script	<code>code_reference/data/build_minority_incidents.py</code>
Documentation	<see project phase docs> + <see project phase docs>

Headline number from this phase

4,110 minority-related Emergency-15 cases identified across Punjab, of which 928 (22.6%) are in Lahore and 312 (7.6%) are strict-label minority-targeted.

Phase 2 — Weeks 2–3 — Exploratory Data Analysis (EDA)

Goal of the phase

Before building any prediction model, understand what the data actually looks like. How does it distribute over time? Across districts? Across communities? Are there obvious patterns? Any data- quality issues?

What we did

Step 2.1 — Ran a standard EDA script

`code_reference/data/run_eda.py` produces:

- **Cases-per-month chart** — total and per-community. Reveals monthly patterns, year-over-year growth, and one-off spikes.
- **Top districts bar chart** — visual ranking of districts by case count. Lahore on top with 22.6%.
- **Top Level-3 categories chart** — most calls fall into “Any Other Religious Issue” (a catch-all), but the more specific sub-categories (“Defiling of Holy Book”, “Hate Speech”, “Attack on Religious Places”) give a flavor of the variety.
- **Community pie chart** — christian dominates the named-community set; unspecified is the largest overall bucket.
- **Lahore top police stations bar** — Mughalpura, Garhi Shahu, Quaid-e-Azam Industrial Estate, Town Ship dominate.
- **Match source bar** — how each row qualified (religious tag, description keyword, both).

Step 2.2 — Built the district severity CSV

Following the original Data Dictionary Report’s specification, we classify each district by case-count band:

Cases	Severity
0–50	Low

Cases	Severity
51–150	Medium
151–300	High
300+	Critical

Result: Lahore and Faisalabad are Critical; Sheikhupura, Sialkot, Gujranwala are High; the rest spread across Medium / Low.

Step 2.3 — Documented findings

<see project phase docs> writes up everything found in this phase with embedded figures and tables.

What was delivered

Artifact	What
EDA script	<code>code_reference/data/run_eda.py</code>
7 figures	<see project figures>
District severity CSV	<see project intermediate data>
Sample-for-review CSV	<see project intermediate data>
Summary JSON	<see project intermediate data>
Documentation	<see project phase docs>

Headline finding

The data shows clear **year-on-year growth** (1,450 → 2,117 → 525 YTD) and clear **geographic concentration** — 5 districts account for the bulk of cases, and within Lahore, 5 police stations account for ~25% of city cases.

Phase 3 — Weeks 3–4 — Interactive Heatmaps

Goal of the phase

Build the **headline visualization** the Research Design Report specifically asks for: a yellow → red heatmap of cases across Punjab, with the densest areas in red.

What we did

Step 3.1 — Chose a tech stack

The original plan said *Folium* (a Python library that wraps Leaflet.js). Folium turned out to not be installed in our environment and `pip install` was blocked by network proxy. We pivoted to a **self-contained Leaflet HTML pattern** — same end result, no Python dependency, the HTML file can be opened directly in any browser.

Step 3.2 — Wrote `code_reference/viz/heatmap.py`

The script reads from `minority_incidents` and produces three HTML files:

- `heatmap_lahore.html` — Lahore-only view (749 mappable cases)
- `heatmap_punjab.html` — All Punjab (3,281 mappable cases)

- `heatmap_strict.html` — Just the 266 strict-label cases that have GPS

Each HTML file: - Embeds the data inline (so it works offline) - Loads Leaflet + Leaflet.heat from CDN (one-time fetch) - Has community-color sub-toggles - Includes a legend

What was delivered

3 self-contained interactive HTML heatmaps in <see project figures>.

Phase 4 — Weeks 5–6 — Feature Engineering

Goal of the phase

For each of the 4,110 cases, compute a set of *features* (measurable quantities) that might help predict whether the case is minority- targeted. The methodology specified 6 families of features.

What we did

Step 4.1 — Built each family one at a time

For each family, we:

1. Created a database schema file (`code_reference/features/schema_*.sql`).
2. Wrote a builder script (`code_reference/features/*.py`).
3. Ran it and verified the numbers.
4. Documented it in <see project phase docs>.

The six feature families

Family	Cols	Source
1. Prior incident density	11	Self-contained from minority_incidents
2. Religious calendar	12	reference_data/religious_calendar.csv (51 events)
3. Political calendar	7	reference_data/political_calendar.csv (19 events)
4. Misinformation events	7	reference_data/misinformation_events.csv (9 events)
5. Police responsiveness	6	Pulled from <source_call_log_table>
6. Social-media sentiment	6	Placeholder — no live feed

Step 4.2 — Wrote the merge script

`code_reference/features/build_features.py` joins all six feature tables on `case_id` and writes the result to `minority_features_merged` — a single 62-column training-ready table with 4,110 rows.

Headline findings from this phase

- **18.9% of cases happen within ± 1 day of a major religious event.** This is the empirical basis for keeping the religious-calendar predictor in the model. About 1 in 5 incidents are calendar-driven.

- The top-5 PSES by max prior-30d case count exactly include the historical clusters from the Research Design Report (Sialkot-Moutra and Faisalabad-Dijkot). Prior incident density is a real signal.

Phase 5 — Weeks 7–8 — Prediction Models

Goal of the phase

Train computer models that can use the 62 features to predict whether a case (or a place $\times$ time) is minority-targeted.

What we built — three models

Model 1 — Per-case classifier (LR + RF)

Question: *“Is this incident, just reported, minority-targeted?”*

- Trained on 2024–2025 (3,576 cases). Tested on 2026 (534 cases).
- Logistic Regression: **ROC AUC 0.841**
- Random Forest: **ROC AUC 0.814, Precision@20 0.35**
- Both meet the methodology’s success thresholds.
- All 4,110 cases scored and written to `minority_predictions` table.
- Model cards: <see project reports>, `_rf.md`.

Model 2 — PS-week forecaster (LR)

Question: *“For each police-station area, will a strict-targeted incident happen there in the next 30 days?”*

- 82,000 (PS $\times$ week) rows. Trained on 2024–2025, tested on 2026.
- LR wins (RF couldn’t handle the sparse positive class).
- **ROC AUC 0.7154** on the test set.
- Top-50 predictions get 7 correct (Precision 14%, vs base rate 0.75% = **18.5 $\times$ better than random**).
- Forward predictions for the next 30 days written to `minority_psweek_forward` — 656 PSES scored.
- This is the **actual early-warning model**.

Model 3 — Volume forecaster (Prophet)

Question: *“How many total minority-related cases should we expect in Punjab next month? In Lahore?”*

- Facebook Prophet time-series library.
- Trained on the post-October-2025 stable window (we detected a regime shift in the data — case volume dropped from $\sim$ 200/mo to $\sim$ 90/mo around Oct 2025 — and decided not to over-extrapolate the trend).
- **Punjab: 9.54% MAPE** on validation — the best precision of the three models.
- Lahore: 26.31% MAPE (lower precision because of small absolute numbers).
- Forecast for next 3 months: $\sim$ 98 cases/mo Punjab, $\sim$ 23 cases/mo Lahore.
- Model card: <see project reports>.

What was delivered

Artifact	What
3 trained models	<project models folder> — LR + RF (per-case) and PS-week LR + Prophet
Metrics JSON	<project reports folder> — one JSON per model with test-set metrics
Model cards	<see project reports>{lr,rf,prophet}.md
Per-case predictions	minority_predictions (4,110 rows)
Forward predictions	minority_psweek_forward (656 rows)
Volume forecast CSVs	<see project intermediate data>{punjab,lahore}.csv

Phase 6 — Weeks 9–10 — The Dashboard

Goal of the phase

Make a tool a human can actually use. Take all the data and model output and put it behind a clickable interface that shows the right information at the right time.

What we built — two versions

Version 1 — Self-contained HTML

app/dashboard.html is a single file you can open in any browser. No setup required. It uses:

- **Leaflet** for the map
- **Chart.js** for the bottom-row charts
- **Embedded JSON data** (all 3,257 cases with model scores)

Features: - Risk-weighted heatmap (default ON) - Top-100 RF-flagged cases as red dots - Click anywhere → popup with nearest cases - **Early Warning panel** showing top-15 PSes for the next 30 days - **Per-case triage panel** with KPI boxes + top-10 cases list - Top-25 PSes by mean RF score (live-recomputing) - Monthly trend chart, community split chart - **Privacy toggle** for showing descriptions (defaults OFF)

Version 2 — Streamlit web app (public deployment)

The Streamlit version sits at app/app.py in the intern handoff bundle. It's a deliberately thin wrapper around the HTML dashboard:

1. Shows a username + password login screen.
2. Checks credentials against an encrypted `secrets.toml` (which is never committed to git).
3. After login, embeds the sanitized HTML dashboard inside the page.

We picked this architecture (embed the HTML rather than re-implement in Python) for three reasons:

- **One source of truth.** The same `dashboard.html` is used for offline review and for public hosting. Edits to the dashboard's look-and-feel happen in one place.
- **Single runtime dependency.** The `requirements.txt` has only one line: `streamlit`. Easier for an intern to spin up.
- **Free hosting fits.** Streamlit Community Cloud reads the repo, installs Streamlit, and serves the app on a `*.streamlit.app` URL. Total monthly cost: \$0.

Version 3 — The “predictions-first” visual rebuild

Late in Phase 6 we did a small but high-impact visual rework, after a review where stakeholders described the dashboard as “*a map of past incidents with stats on the side.*” — the opposite of what an *early-warning* system should look like. The fix:

1. **New header KPI.** Replaced the technical “RF flagged 0.5: N” badge with “ *N police-stations flagged for the next 30 days · Top: · score X.XX*”. The first thing the eye reads is forward-looking.
2. **Forecast layer on the map.** Top-15 forecast police stations now draw as orange highlighted circles *above* the historical heatmap. The map itself tells the forecast story.
3. **One-sentence subtitle** under the title: “*Forecasts where minority-targeted incidents are most likely in the next 30 days.*”

The dashboard’s visual hierarchy now says “forecast” at every level a stakeholder’s eye lands. Historical data is the supporting layer.

Phase 7 — Weeks 11–12 — Documentation + Delivery

Goal of the phase

Turn the working system into a *deliverable*. Anyone receiving this folder — whether technical or not — should be able to (a) understand what it is, (b) run it, and (c) trust what it says.

What we did

Step 7.1 — System audit

Wrote <see project audit doc> — 12-section verification document. Every file existence, DB table row count, model metric, privacy property, and reproducibility check. Result: clean GO for external presentation.

Step 7.2 — Privacy hardening

Reviewed every dashboard view. Identified that descriptions contained phone numbers, addresses, names, and call-center metadata. Built an auto-redactor that strips: - Phone numbers (Pakistan formats) - “Contact No:” fields - CNICs (13-digit IDs) - House numbers (H#363) - Street numbers (ST#8) - SCC call-center metadata blocks - **Name:** field

Added a “**Show description**” toggle to the dashboard that defaults OFF for external presentations.

Step 7.3 — Portable bundle

Wrote OFFLINE.md and QUICKSTART.md explaining how to run the project on any laptop. Wrote code_reference/data/export_snapshot.py to dump all 11 DB tables to CSV (<see project CSV exports>). Packaged everything into the full project archive (25 MB).

Step 7.4a — Public deployment + intern handoff

Built a separate, slimmer artifact aimed at the intern continuing the project, called minorities_ews_intern_handoff/. It is a clean GitHub-ready folder structured around three top-level subfolders:

- app/ — the Streamlit dashboard (sanitised), requirements.txt, deploy guide, login secrets template, sanitise script.
- docs/ — the six final deliverables in .md, .pdf, and .docx.

- `code_reference/` — the original Python source for study (no DB credentials, no raw data).

A `scripts/sanitize.py` script does two passes on the source HTML dashboard:

1. **PII pass.** Every `"desc": "..."` field in the embedded JSON is replaced with a placeholder. Result: zero caller free-text in the public build.
2. **Polish pass.** Strips raw row-count totals and technical model- metric captions from the header (these are correct in the report but noisy in a live demo). Adds the forecast subtitle, the forecast-focused header KPI, and the forecast map layer.

The handoff folder is designed so a student can `git init`, push to GitHub, point Streamlit Community Cloud at `app/app.py`, set a password in the Secrets pane, and have a live public URL in under ten minutes.

Step 7.4 — Five formal deliverables

This document is one of the five. The full set:

#	Title	Length	Audience
1	Policy Brief	2 pages	Senior leadership, stakeholders
2	Detailed Report	10–15 pages	Academic, technical reviewer
3	Research Design Report (updated)	~7 pages	Methodologists, researchers
4	Data Dictionary	~6 pages	Database administrators, analysts
5	Project Timeline (this document)	~5 pages	Reviewers, future implementers

Each is delivered in **three formats**: - Markdown (`.md`) — editable, version-controllable - Microsoft Word (`.docx`) — editable in Word / Google Docs - PDF (`.pdf`) — print-ready, shareable

All five are in `outputs/deliverables/`.

What was delivered

Artifact	What
System audit	<see project audit doc>
Privacy hardening	Updated <code>code_reference/viz/dashboard.py</code> + dashboard with toggle
Bundle	the full project archive (28 MB, 142 files)
5 deliverables × 3 formats	<code>outputs/deliverables/*.md, .docx, .pdf</code>
Offline guide	<code>OFFLINE.md</code>
Quickstart	<code>QUICKSTART.md</code>
Claude context	<code>CLAUDE.md</code> (for future Claude sessions)

Cross-Reference — What File Came From Which Phase

If you want to know about...	Read this
The base 4,110-case table	Phase 1
What the data looks like in aggregate	Phase 2
Maps + visual exploration	Phase 3
Feature engineering	Phase 4 (Weeks 5–6)

If you want to know about...	Read this
The three prediction models	Phase 5 (Weeks 7–8)
The dashboard you click on	Phase 6 (Weeks 9–10)
Privacy + delivery + this document	Phase 7 (Weeks 11–12)

What’s Been Delivered

Every item below is a concrete artifact produced during the project:

- **4,110 cases** in the working incidents table
- **62 features** in the merged training table
- **3 models** trained — per-case classifier, PS-week forecaster, Prophet volume forecaster — each meeting its success threshold
- **Interactive dashboard** in both self-contained HTML and gated Streamlit forms
- **Six deliverable documents** in .md, .docx, and .pdf formats
- **Portable project archive** ready for handover

Every number cited in this document traces back to a verified script output or database query.

*This document is the updated edition of the original Timeline plan, with each phase now backed by concrete artifacts. For the rest of the project, start with the handoff **README.md**.*